

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	JAGADISH.V	Reg. No.:	192521352
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Feedback Vertex Set Problem (NPComplete Problem)

Find a minimum set of vertices whose removal makes the graph acyclic. Clearly define constraints related to cycle elimination. Analyze how constraints impact feasible solutions. Develop a backtracking-based solution. Apply pruning techniques to eliminate unnecessary vertex removals. Justify correctness and discuss NP-Complete classification.

Code of Conduct

I JAGADISH.V 192521352(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jagadish.v

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Feedback Vertex Set using Backtracking and Pruning

Step-by-Step Algorithm

1. **Start**
2. Read the graph and store its vertices and edges.
3. Check whether the current graph contains a cycle.
4. If the graph is already acyclic:
 - Return the current removed-vertex set.
5. If a cycle exists:
 - Select a vertex from the graph as a candidate for removal.
6. Create two branches:
 - **Include:** remove the selected vertex.
 - **Exclude:** keep the selected vertex.
7. Recursively solve both branches.
8. Apply **pruning**:
 - If the current removed set is already larger than the best solution found, stop exploring that branch.
9. Compare the solutions obtained from both branches.
10. Keep the solution with the **minimum number of removed vertices**.
11. Continue until all possible necessary choices have been considered.
12. Return the minimum Feedback Vertex Set.
13. **Stop.**

Pseudocode

Algorithm FeedbackVertexSet(G, removed)

if G is acyclic then

 return removed

if size(removed) $\geq$ best_solution_size then

 return

choose a vertex v involved in a cycle

// Branch 1: Remove v

$G1 \leftarrow G - v$

solution1 $\leftarrow$ FeedbackVertexSet($G1$, removed $\cup \{v\}$)

// Branch 2: Keep v

solution2 $\leftarrow$ FeedbackVertexSet(G, removed)

return smaller(solution1, solution2)

End Algorithm

QUESTIONS

Q1

Feedback Vertex Set Problem
(NP-Complete Problem)
100 marks

0/1 answered

Q1 Feedback Vertex Set Problem (NP-Complete Problem)

100 marks

Find a minimum set of vertices whose removal makes the graph acyclic. Clearly define constraints related to cycle elimination. Analyze how constraints impact feasible solutions. Develop a backtracking-based solution. Apply pruning techniques to eliminate unnecessary vertex removals. Justify correctness and discuss NP-Complete classification.

Your Code

```
1 from itertools import combinations
2
3 def is_acyclic(adj, nodes):
4     visited, stack = set(), set()
5     def has_cycle(v):
6         visited.add(v)
7         stack.add(v)
8         for neighbor in adj.get(v, []):
9             if neighbor in nodes:
10                 if neighbor not in visited:
11                     if has_cycle(neighbor): return True
12                 elif neighbor in stack: return True
13             stack.remove(v)
14         return False
15
16 for node in nodes:
17     if node not in visited:
18         if has_cycle(node): return True
```

Input

stdin input

Output

Min FVS: [0]